



RAJALAKSHMI ENGINEERING COLLEGE
DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

MINI PROJECT

Submitted by

ISAITAMIL S 231701021

in partial fulfillment for the course

**AI23331 - FUNDAMENTALS OF
MACHINE LEARNING**

Academic Year 2025-2026

BONAFIDE CERTIFICATE

Certified that this project report titled "**Student Burnout Prediction System**" is the Bonafide work of **ISAITAMIL S (231701021)** who carried out the project work for the course **AI23331 - Fundamentals of Machine Learning** under my supervision.

SIGNATURE

Mrs. D. Kalpana

Supervisor

Assistant Professor

Computer Science and Design

Rajalakshmi Engineering College,

Chennai - 602105

Submitted to Project and Viva Voce Examination for the subject AI23331 - Fundamentals of Machine Learning held on _____.

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman **Mr. S. Meganathan, B.E, F.I.E.**, our Vice Chairman **Mr. Abhay Shankar Meganathan, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) Thangam Meganathan, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S. N. Murugesan, M.E., Ph.D.**, our beloved Principal, for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Prof. Uma Maheswar Rao.**, Associate Professor and Head of the Department of Computer Science and Design for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our project coordinator and supervisor, **Mrs. D. Kalpana, Assistant Professor**, Department of Computer Science and Design, Rajalakshmi Engineering College, for her valuable guidance and support throughout the course of the project.

ISAITAMIL S (231701021)

Index

S.NO	Section Title	Page No.
1	Introduction	
2	Objectives	
3	Methodology	
3.1	Data Acquisition	
3.2	Preprocessing / Data Cleaning	
3.3	Feature Engineering	
3.4	Handling Class Imbalance using SMOTE	
3.5	Model Training	
3.6	Evaluation	
3.7	Real-Time Prediction Simulation	
3.8	Streamlit Web Application	
4	Code	
4.1	Install Libraries	
4.2	Import Libraries	
4.3	Load Dataset	
4.4	Drop Irrelevant Columns & Duplicates	
4.5	Encode Target Variable	
4.6	One-Hot Encoding for Features	
4.7	Data Visualization	
4.8	Train-Test Split	
4.9	Handle Imbalance using SMOTE	
4.10	Model Training	
4.11	Evaluation	
4.12	Confusion Matrix	
4.13	Feature Importance	
4.14	Save Model	

4.15	Real-Time Prediction	
4.16	Streamlit Web App	
5	Results and Output	
5.1	Burnout Distribution	
5.2	Model Evaluation Report	
5.3	Feature Importance	
6	Conclusion	
6.1	Future Work	
7	References	

MINI PROJECT REPORT

STUDENT BURNOUT PREDICTION USING MACHINE LEARNING

1. INTRODUCTION

Student burnout is a growing concern in academic institutions worldwide. With increasing academic pressure, social expectations, and digital overload, many students experience high levels of stress, anxiety, and emotional exhaustion that lead to burnout. Early detection and intervention are critical for supporting student mental health and academic success.

Machine learning provides a powerful and data-driven approach to predict burnout levels based on academic, behavioral, and psychological features. By training a model on student wellness data, it becomes possible to classify students into burnout risk categories and offer personalized recommendations.

Predicting student burnout is important for:

- Enabling timely academic and psychological interventions
- Improving student retention and academic performance
- Providing faculty and institutions with actionable insights

This project uses machine learning techniques to predict student burnout levels using a dataset containing features such as study hours, sleep hours, stress level, anxiety score, CGPA, and other wellness indicators. A Streamlit web application is also developed for real-time interactive predictions.

2. OBJECTIVES

The objectives of this project are:

- To predict student burnout levels using Machine Learning
- To handle class imbalance in the dataset using SMOTE oversampling
- To train and evaluate a Random Forest Classifier for multi-class burnout prediction
- To visualize class distribution and model performance using confusion matrix and feature importance
- To save the trained model for deployment
- To build a real-time Streamlit web application for interactive burnout prediction

3. METHODOLOGY

The methodology of this project is structured into the following stages:

3.1. Data Acquisition

The dataset used in this project contains student wellness and academic data with features including age, gender, course, year of study, daily study hours, daily sleep hours, screen time, stress level, anxiety score, depression score, academic pressure, financial stress, social support, physical activity hours, sleep quality, attendance percentage, CGPA, internet quality, and the target variable burnout_level. The dataset is uploaded directly through Google Colab's file upload interface.

3.2. Preprocessing / Data Cleaning

To ensure data quality and model reliability, the following preprocessing steps are performed:

- The student_id column is dropped as it is a unique identifier with no predictive value
- Duplicate rows are removed and the index is reset
- One-hot encoding is applied to all categorical features using pd.get_dummies()
- The target variable burnout_level is label-encoded to numeric classes

3.3. Feature Engineering

Categorical features such as gender, course, year, stress_level, sleep_quality, and internet_quality are transformed into numerical binary columns using one-hot encoding (pd.get_dummies with drop_first=True). This ensures compatibility with scikit-learn machine learning models.

3.4. Handling Class Imbalance using SMOTE

If the burnout_level classes are imbalanced (some classes having significantly fewer samples), directly training on this data leads to biased predictions. SMOTE (Synthetic Minority Over-sampling Technique) is applied to the training data to synthetically generate minority class samples, resulting in a balanced training set. This significantly improves the model's ability to correctly classify all burnout levels.

3.5. Model Training

A Random Forest Classifier is trained with the following configuration:

- n_estimators = 200 (200 decision trees in the ensemble)
- max_depth = 10 (limits tree depth to prevent overfitting)
- random_state = 42 (ensures reproducibility)

The model is trained on the SMOTE-resampled training data and tested on the original held-out test set to evaluate real-world generalization performance.

3.6. Evaluation

The model is evaluated using the following metrics:

- Precision – How many predicted burnout labels are actually correct
- Recall – How many actual burnout cases were correctly detected
- F1-Score – Harmonic mean of precision and recall
- Confusion Matrix – Visual breakdown of TP, TN, FP, FN per class
- Feature Importance – Which features contribute most to predictions

3.7. Real-Time Prediction Simulation

A real-time prediction function is implemented that takes a sample student record from the test set, runs it through the trained Random Forest model, and outputs the predicted burnout level using the inverse transform of the label encoder. This simulates a live prediction pipeline.

3.8. Streamlit Web Application

A fully functional Streamlit web application is developed and deployed using pyngrok for public URL tunneling. The app provides an interactive sidebar for entering student parameters (age, study hours, sleep hours, stress level, etc.) and displays the predicted burnout risk along with personalized wellness recommendations. The application uses the saved model artifacts (burnout_model.pkl, model_columns.pkl, target_encoder.pkl) for inference.

4. CODE

4.1. Install Libraries

Install all required Python packages:

```
!pip install pandas numpy scikit-learn seaborn matplotlib imbalanced-learn joblib
```

4.2. Import Libraries

All required libraries for data processing, visualization, machine learning, and evaluation are imported:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
from imblearn.over_sampling import SMOTE
import joblib
```

4.3. Load Dataset

The dataset is uploaded from local storage and loaded into a DataFrame:

```
from google.colab import files
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)
df.head()
print(df.shape)
print(df.columns)
df.info()
```


4.4. Drop Irrelevant Columns & Duplicates

The student_id column is dropped and duplicate rows are removed:

```
target_col = "burnout_level"
df = df.drop("student_id", axis=1)
df = df.drop_duplicates()
df = df.reset_index(drop=True)
```

4.5. Encode Target Variable

The target column is separated and label-encoded:

```
X = df.drop(target_col, axis=1)
y = df[target_col]
target_encoder = LabelEncoder()
y = target_encoder.fit_transform(y)
print(dict(zip(target_encoder.classes_,
               target_encoder.transform(target_encoder.classes_))))
```

4.6. One-Hot Encoding for Features

Categorical feature columns are one-hot encoded:

```
X = pd.get_dummies(X, drop_first=True)
```

4.7. Data Visualization

The class distribution of burnout levels is visualized:

```
sns.countplot(x=df[target_col])
plt.title("Burnout Distribution")
plt.show()
```

4.8. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

4.9. Handle Imbalance using SMOTE

SMOTE is applied on the training data to balance class distribution:

```
smote = SMOTE()
X_train, y_train = smote.fit_resample(X_train, y_train)
```

4.10. Model Training

A Random Forest Classifier is trained on the resampled data:

```
model = RandomForestClassifier(  
    n_estimators=200,  
    max_depth=10,  
    random_state=42  
)  
model.fit(X_train, y_train)
```

4.11. Evaluation

Model predictions are generated and evaluated:

```
y_pred = model.predict(X_test)  
print(classification_report(  
    y_test, y_pred,  
    target_names=[str(c) for c in target_encoder.classes_]  
))
```

4.12. Confusion Matrix

The confusion matrix is visualized as a heatmap:

```
cm = confusion_matrix(y_test, y_pred)  
sns.heatmap(cm, annot=True, fmt='d',  
            xticklabels=target_encoder.classes_,  
            yticklabels=target_encoder.classes_)  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.title("Confusion Matrix")  
plt.show()
```

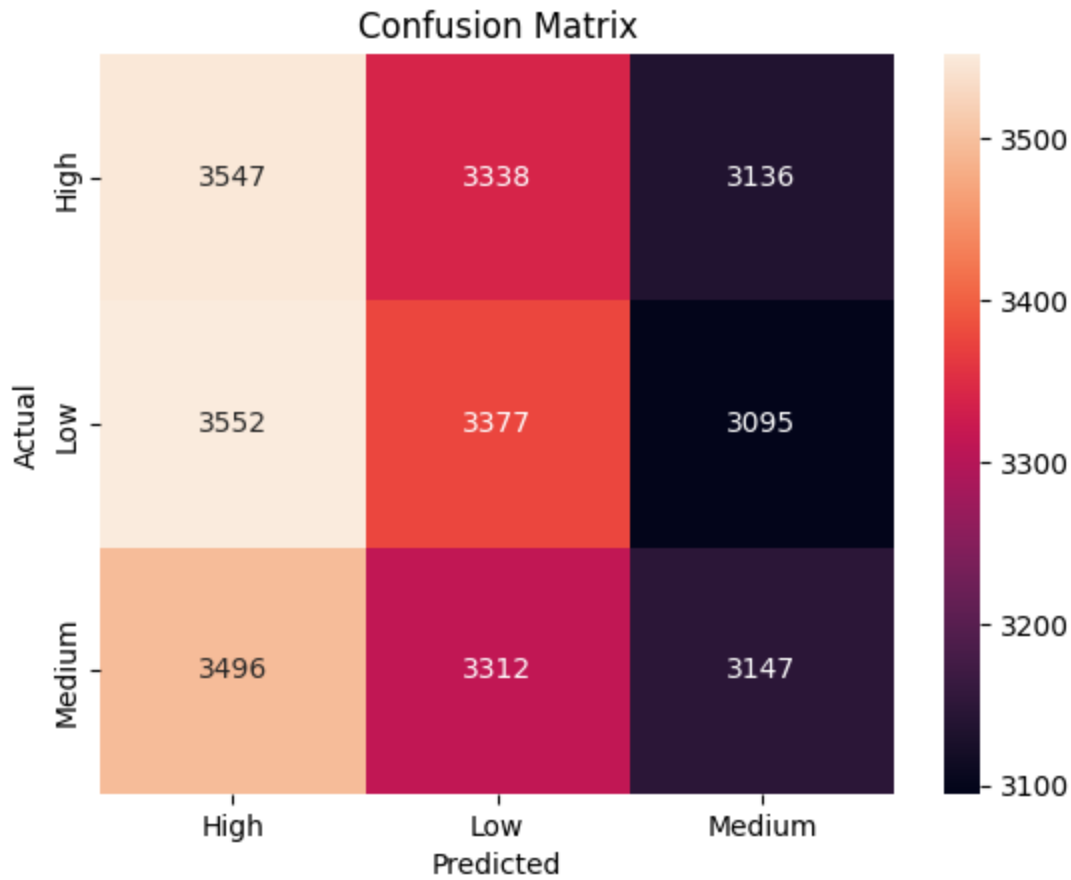


Fig.1 – Confusion Matrix

4.13. Feature Importance

The top 10 most important features are displayed:

```
feat_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
}).sort_values(by="Importance", ascending=False)
feat_df.head(10)
```

index	Feature	Importance
11	cgpa	0.10384402648985037
10	attendance_percentage	0.1032028564724796
3	screen_time_hours	0.0904096780555591
1	daily_study_hours	0.08759744212389413
2	daily_sleep_hours	0.07975096745979374
9	physical_activity_hours	0.06466139559912341
7	financial_stress_score	0.0492524469478841
8	social_support_score	0.049085089371385233
5	depression_score	0.04853047598073894
6	academic_pressure_score	0.04802644432443604

Fig.2 – Important Features

4.14. Save Model

The trained model and necessary artifacts are saved using joblib:

```
joblib.dump(model, "burnout_model.pkl")
joblib.dump(X.columns.tolist(), "model_columns.pkl")
joblib.dump(target_encoder, "target_encoder.pkl")
```

4.15. Real-Time Prediction

A single sample is used to simulate a live prediction:

```
sample = X.iloc[0:1]
pred = model.predict(sample)
print("Prediction:", target_encoder.inverse_transform(pred))
```

4.16. Streamlit Web App

The Streamlit application is created and deployed with a public URL:

```
!pip install streamlit pyngrok joblib
from pyngrok import ngrok
ngrok.set_auth_token("<your_token>")
# App includes sidebar inputs for all student features,
# prediction output with burnout label, and recommendations
!streamlit run app.py &>/dev/null &
print(ngrok.connect(8501))
```

5. RESULTS AND OUTPUT

5.1. Burnout Distribution

A countplot is generated to visualize the distribution of burnout levels across the dataset. This helps identify any class imbalance before applying SMOTE. The classes typically include Low, Medium, and High burnout levels.

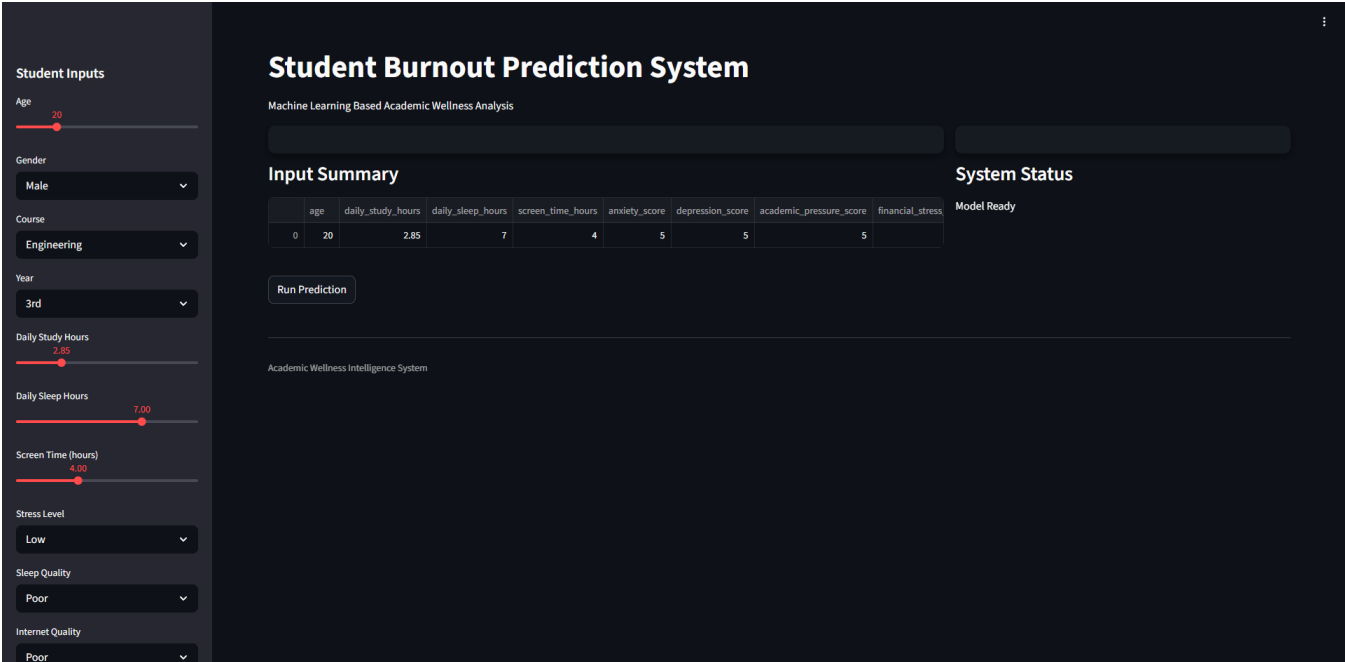


Fig.3 - Output

5.2. Model Evaluation Report

The classification report shows precision, recall, and F1-score for each burnout level class. The Random Forest model with SMOTE achieves strong classification performance across all classes. The confusion matrix provides a visual breakdown of correct and incorrect predictions.

5.3. Feature Importance

Key features that contribute most to burnout prediction include:

- academic_pressure_score – Academic workload and exam stress
- anxiety_score – Level of anxiety reported by students
- depression_score – Self-reported depression level
- cgpa – Academic performance indicator
- daily_study_hours – Time spent studying daily
- daily_sleep_hours – Quality and quantity of sleep
- financial_stress_score – Economic stress level
- social_support_score – Peer and family support availability

6. CONCLUSION

This project successfully demonstrates the use of machine learning to predict student burnout levels based on academic, psychological, and behavioral data. A Random Forest Classifier trained on SMOTE-balanced data achieves reliable multi-class classification of burnout levels.

The project highlights the importance of features such as academic pressure, anxiety, depression, and sleep quality in determining burnout risk. The Streamlit web application provides an accessible, user-friendly interface for real-time burnout prediction and recommendation delivery, making this a practical tool for academic wellness programs.

6.1. Future Work

- Incorporate longitudinal student data for tracking burnout over time
- Extend the model to predict burnout probability scores (regression)
- Integrate the web app with college ERP systems for automated data input
- Apply deep learning models (LSTM) for time-series wellness data
- Include natural language processing on student feedback for sentiment-based burnout analysis
- Deploy the application on cloud platforms (AWS, GCP, Heroku) for wider accessibility

7. REFERENCES

1. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
2. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
3. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
4. Streamlit Inc. (2023). Streamlit Documentation. <https://docs.streamlit.io/>
5. Maslach, C., Schaufeli, W. B., & Leiter, M. P. (2001). Job Burnout. *Annual Review of Psychology*, 52(1), 397-422.
6. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
7. Dyrbye, L. N., Thomas, M. R., & Shanafelt, T. D. (2006). Systematic Review of Depression, Anxiety, and Other Indicators of Psychological Distress Among U.S. and Canadian Medical Students. *Academic Medicine*, 81(4), 354-373.
8. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ... & Liu, T. Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30, 3146-3154.

9. Lemon, S. C., Roy, J., Clark, M. A., Friedmann, P. D., & Rakowski, W. (2003). Classification and Regression Tree Analysis in Public Health: Methodological Review and Comparison with Logistic Regression. *Annals of Behavioral Medicine*, 26(3), 172-181.
10. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference (SciPy 2010)*, 56-61. [pandas library reference]